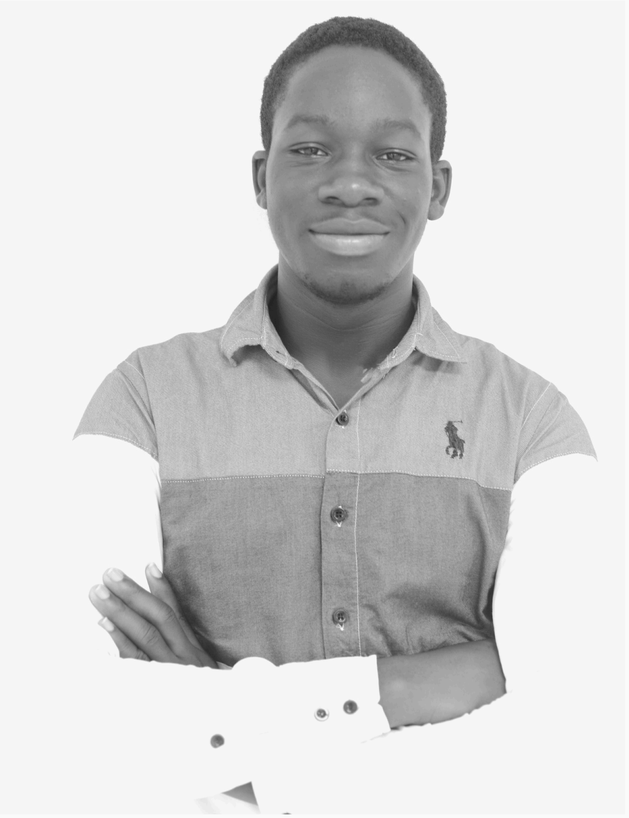
Olabissi M. SANNI

Cours HTML, CSS, JavaScript

L1IRT- ESGIS

2024-2025



20H pour un Portfolio et
80 QCM en 1H30min

HTML, CSS, JAVASCRIPT

Objectifs du Cours

Durant les 20h de cours on va parcourir les notions de base en HTML, CSS, JAVASCRIPT. A la fin du cours vous produiserez chacun un portfolio de vous ou de l'un de vos parent ou tuteur en utilisant les notions apprises.



Objectif défini

Comprendre et connaître :

- l'historique de l'HTML
- ce que c'est une balise
- ce que c'est un attribut
- quelle est la structure d'un fichier HTML
- la création d'un liens interne ou externe
- la création et manipulation d'un formulaire
- les bonnes pratiques de l'HTML



Objectif défini

Comprendre et connaître :

- Ce que c'est le CSS
- Différence entre HTML et CSS.
- Méthodes d'intégration de CSS
- Syntaxe de base
- Sélecteurs CSS
- Propriétés CSS de base
- Mise en page et disposition
- Styles avancés et design responsive
- Concepts avancés et bonnes pratiques CSS



Objectif défini

Comprendre et connaître :

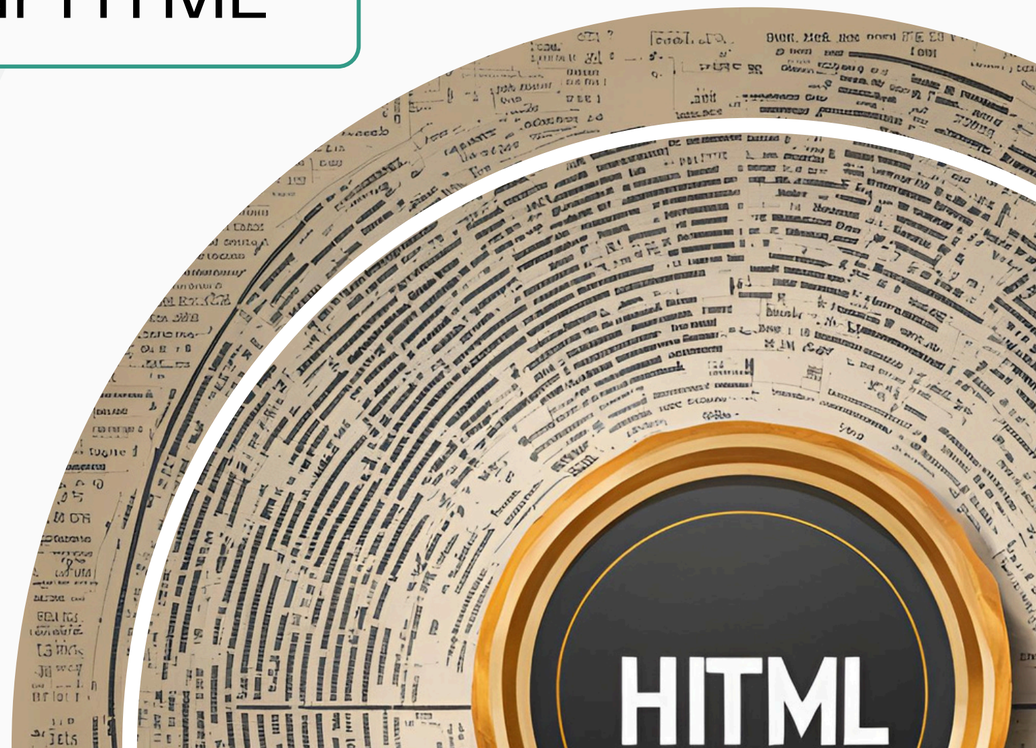
- l'historique de JavaScript
- Variables, types de données et opérateurs
- Structures de contrôle
- Fonctions en JavaScript
- Le DOM (Document Object Model)
- Gestion des événements
- Manipulation des données avec JavaScript
- Concepts avancés de JavaScript
- Bonnes pratiques et outils de développement



1

HTML

8h de discussion sur HTML



Plan HTML



Comprendre et connaître :

- l'historique de l'HTML
- ce que c'est une balise
- ce que c'est un attribut
- quelle est la structure d'un fichier HTML
- la création d'un liens interne ou externe
- la création et manipulation d'un formulaire
- les bonnes pratiques de l'HTML

Introduction

Historique de HTML



Question 1 :

Qu'est-ce que c'est le HTML??

Comprendre que HTML (HyperText Markup Language) est le langage de base pour créer des pages web. Il structure le contenu de la page et permet aux navigateurs de comprendre comment afficher les éléments.

Question 2 :

Qu'est est l'historique de HTML pour qu'on arrive à HTML5?

Voici un aperçu de l'histoire et de l'évolution du HTML :

1. Les Origines (1989 – 1991)

- Création par Tim Berners-Lee : Le HTML a été inventé par Tim Berners-Lee, un informaticien britannique travaillant au CERN (Organisation Européenne pour la Recherche Nucléaire). En 1989, il a proposé un projet de gestion de l'information pour permettre aux chercheurs de partager facilement des documents. C'est ainsi qu'est née la première version du HTML.
- Premier site web (1991) : En 1991, Tim Berners-Lee a publié le premier site web, qui servait de guide sur l'utilisation du World Wide Web (WWW). Le HTML 1.0 était alors très rudimentaire et comportait un nombre limité de balises, comme <p>, <a>, <img>, <h1> – <h6>, et <ul>.

Introduction

Historique de HTML

Question 2 :

Qu’est est l’historique de HTML pour qu’on arrive à HMLT5?

2. HTML 2.0 (1995)
   - Standardisation : Après le succès initial du HTML, il est devenu nécessaire de créer une version standardisée pour garantir que les documents web puissent être interprétés de manière uniforme par les navigateurs. HTML 2.0 a été la première version officielle standardisée par le World Wide Web Consortium (W3C) en 1995.
   - Caractéristiques : Cette version contenait les fonctionnalités du HTML original, ainsi que des améliorations mineures et des balises supplémentaires pour mieux structurer le texte, créer des formulaires, et gérer les images et les liens.
3. HTML 3.2 et HTML 4.0 (1997 – 1999)
   - HTML 3.2 (1997) : Avec l'essor d'Internet dans les années 90, il y avait une forte demande pour ajouter plus de fonctionnalités. HTML 3.2, publié par le W3C en 1997, a introduit des tableaux (<table>), des applets Java et d'autres éléments interactifs. Cependant, elle manquait de certaines fonctionnalités proposées par le HTML+ (une version non officielle antérieure).

Introduction

Historique de HTML

Question 2 :

Qu’est est l’historique de HTML pour qu’on arrive à HMLT5?

- HTML 4.0 (1999) : HTML 4.0 a marqué une étape majeure dans l'évolution du langage. Il a introduit de nombreuses nouvelles balises et fonctionnalités, y compris :
  - Amélioration de la structuration des documents : Introduction des balises et pour organiser le contenu.
  - Amélioration des formulaires : Nouvelles options pour les éléments de formulaire et meilleure gestion des entrées utilisateur.
  - Support des feuilles de style (CSS) : HTML 4.0 a encouragé la séparation de la structure (HTML) et de la présentation (CSS), permettant des pages plus esthétiques et maintenables.

4. L'ère XHTML (2000 – 2008)

- XHTML 1.0 (2000) : XHTML (eXtensible HyperText Markup Language) était une version reformulée du HTML en tant que langage XML, plus strict et plus propre. L'objectif était de créer des documents plus rigoureux, avec une syntaxe stricte. Chaque balise devait être correctement fermée, et le code devait être bien structuré.
- Problèmes d'adoption : Bien que XHTML soit plus rigoureux, il a été difficilement adopté par les développeurs en raison de sa rigidité, et il a conduit à des problèmes de compatibilité avec certains navigateurs.

Introduction

Historique de HTML

Question 2 :

Qu’est est l’historique de HTML pour qu’on arrive à HMLT5?

5. HTML5 (2014 – Aujourd'hui)

- HTML5 (2014) : Le W3C a travaillé avec le WHATWG (Web Hypertext Application Technology Working Group) pour créer une nouvelle version du HTML qui répondait aux besoins modernes du web. HTML5 est officiellement sorti en octobre 2014 après plusieurs années de développement.
- Caractéristiques clés :
  - Balises sémantiques : Introduction de balises comme <header>, <footer>, <article>, <section>, <nav>, et plus, pour rendre les pages web plus sémantiques et accessibles.
  - Multimédia amélioré : Support direct pour les vidéos et les audios avec <video> et <audio>, sans besoin de plugins externes (comme Flash).
  - API intégrées : HTML5 a introduit de nombreuses API JavaScript, comme la géolocalisation, le stockage local (localStorage et sessionStorage), et la gestion de contenu dynamique avec l'API Canvas (<canvas>).
  - Formulaires améliorés : Nouvelles options pour créer des formulaires plus interactifs et fonctionnels avec des types d'entrées comme email, date, color, range, etc.
  - Responsive Design : Meilleure gestion pour les appareils mobiles, permettant aux développeurs de créer des sites web adaptatifs avec plus de facilité.

Balises, Attributs

Balises, Attributs

Question 1 :

Qu’est est ce que c’est une balise et quelles sont les types de balises qui existes?

En HTML, une balise est un élément qui sert à structurer et organiser le contenu d'une page web. Elle est entourée de chevrons (< >) et permet de définir la nature ou le rôle du contenu situé entre les balises ouvrantes et fermantes ou juste une balise orpheline ou auto-fermante.
Les balises sont l'élément de base de la syntaxe HTML, et chaque type de balise a un rôle spécifique (titres, paragraphes, images, liens, etc.).

Types de balises en HTML

Les balises HTML peuvent être classées en différentes catégories selon leur fonction et leur comportement. Voici les principaux types :

1. Balises de structure

Ces balises définissent la structure de base d'une page HTML et aident à organiser le contenu de manière logique.

- <html> : La balise racine de chaque document HTML, contenant tout le contenu de la page.
- <head> : Contient des métadonnées, des liens vers des feuilles de style, et des scripts.
- <body> : Contient le contenu visible de la page web (texte, images, vidéos, etc.).
- <header>, <footer>, <main>, <section>, <article>, <aside>, <nav> : Balises sémantiques introduites avec HTML5 pour organiser le contenu de manière significative.

Balises, Attributs

Balises, Attributs

Question 1 :

Qu’est est ce que c’est une balise et quelles sont les types de balises qui existes?

2. Balises de texte

Ces balises sont utilisées pour structurer et formater le texte.

- <h1>, <h2>, ..., <h6> : Balises de titres, <h1> étant le plus important et <h6> le moins.
- <p> : Crée des paragraphes de texte.
- , <b> : Affiche du texte en gras. a une signification sémantique supplémentaire pour les lecteurs d'écran.
- , <i> : Affiche du texte en italique. ajoute une signification sémantique (emphase) au texte.
- <blockquote> : Pour insérer des citations longues.
- <pre> : Affiche du texte préformaté avec des espaces et des sauts de ligne respectés.

3. Balises de média

Ces balises permettent d'intégrer des éléments multimédias comme des images, des vidéos et des audios.

- <img> : Pour insérer des images. Cette balise est auto-fermante et n'a pas besoin d'une balise de fermeture.
- <audio> : Pour intégrer des fichiers audio.
- <video> : Pour insérer des vidéos.
- <iframe> : Pour intégrer du contenu externe comme des vidéos YouTube ou des cartes Google Maps.

Balises, Attributs

Balises, Attributs

Question 1 :

Qu’est est ce que c’est une balise et quelles sont les types de balises qui existes?

4. Balises de liens et navigation

Ces balises sont utilisées pour la navigation entre différentes pages ou sections.

- <a> : Crée des liens hypertextes.
- <nav> : Utilisé pour contenir des groupes de liens de navigation.

5. Balises de formulaires

Ces balises permettent de créer des formulaires pour interagir avec les utilisateurs.

- <form> : Délimite un formulaire.
- <input> : Crée des champs de saisie (textes, mots de passe, boutons, etc.).
- <textarea> : Pour des zones de texte plus grandes.
- <button> : Crée un bouton cliquable.
- <select>, <option> : Crée des menus déroulants.
- <label> : Associe une description à un élément de formulaire.

6. Balises de conteneur et de sectionnement

Ces balises aident à organiser le contenu en blocs ou en sections spécifiques.

- : Balise de conteneur générique pour organiser le contenu en blocs.
- : Balise de conteneur en ligne pour appliquer du style à une partie du texte.
- <section>, <article>, <aside> : Utilisées pour organiser le contenu en sections sémantiques spécifiques.

Balises, Attributs

Balises, Attributs

Question 1 :

Qu’est est ce que c’est une balise et quelles sont les types de balises qui existes?

7. Balises de tableaux

Ces balises permettent de créer des tableaux pour organiser les données en lignes et colonnes.

- <table> : Crée un tableau.
- <tr> : Définit une ligne de tableau.
- <td> : Définit une cellule de données.
- <th> : Définit une cellule d'en-tête de tableau.
- <caption> : Ajoute une légende au tableau.

8. Balises sémantiques

Les balises sémantiques ont été introduites avec HTML5 pour donner un sens plus clair au contenu des pages. Elles aident les navigateurs et les moteurs de recherche à comprendre la structure et l'importance des différentes parties du contenu.

- <header> : En-tête de section ou de page.
- <footer> : Pied de page ou de section.
- <article> : Contenu indépendant et autonome.
- <aside> : Contenu lié ou complémentaire (comme une barre latérale).
- <section> : Section thématique du contenu.
- <figure> et <figcaption> : Contenu visuel avec légendes.

Balises, Attributs

Balises, Attributs

Question 1 :

Qu’est est ce que c’est une balise et quelles sont les types de balises qui existes?

9. Balises auto-fermantes

Certaines balises ne nécessitent pas de balise fermante car elles ne contiennent pas de contenu. Elles sont dites auto-fermantes ou autonomes.

- <img> : Pour afficher des images.
-
 : Insère un saut de ligne.
- <hr> : Insère une ligne horizontale.
- <input> : Crée un champ de saisie.

Les balises HTML sont essentielles pour structurer et organiser le contenu sur le web. Elles permettent aux développeurs de définir la disposition des éléments, de formater le texte, d'insérer des images, de créer des formulaires et bien plus encore. Chaque type de balise a un rôle spécifique, et comprendre leur utilisation est crucial pour créer des pages web efficaces et bien structurées.

Balises, Attributs

Balises, Attributs

Question 2 :

Qu’est est ce que c’est un attribut et quelles sont les types d’attributs qui existes?

Un attribut est une information supplémentaire ajoutée à une balise HTML pour en définir les propriétés ou fournir des informations supplémentaires sur cette balise. Les attributs se trouvent à l'intérieur de la balise ouvrante et prennent généralement la forme suivante :

<balise attribut="valeur">Contenu</balise>

Par exemple, pour ajouter un lien hypertexte avec un texte cliquable, on utilise l'attribut href :

Cliquez ici pour visiter Example

Dans cet exemple :

- href est un attribut de la balise <a>.
- https://www.example.com est la valeur de cet attribut.
- Cliquez ici pour visiter Example est le contenu visible par l'utilisateur.

Les attributs peuvent modifier le comportement ou l'apparence d'un élément HTML et sont souvent utilisés pour des choses comme définir des liens, ajouter des styles CSS, spécifier des valeurs par défaut pour les champs de formulaire, etc.

Balises, Attributs

Balises, Attributs

Question 2 :

Qu’est est ce que c’est un attribut et quelles sont les types d’attributs qui existes?

Types d'attributs en HTML

Voici quelques-uns des types d'attributs les plus couramment utilisés en HTML :

1. Attributs de base
   - id : Identifie de manière unique un élément sur une page. Utile pour les styles CSS ou pour la manipulation avec JavaScript.
   - Ceci est un paragraphe.
   - class : Attribue une ou plusieurs classes à un élément, ce qui permet d'appliquer des styles CSS ou de manipuler plusieurs éléments avec JavaScript.
   - Contenu de la boîte rouge
2. Attributs pour les liens et les images
   - href : Définit l'URL vers laquelle le lien doit diriger.
   - Visiter Google
   - target : Indique où ouvrir le lien (_self, _blank, _parent, _top).
   - Ouvrir dans un nouvel onglet
   - src : Indique la source d'une image ou d'un fichier multimédia.
   - <img src="image.jpg" alt="Description de l'image">

Balises, Attributs

Balises, Attributs

Question 2 :

Qu’est est ce que c’est un attribut et quelles sont les types d’attributs qui existes?

- alt : Fournit une description alternative d'une image, utile pour l'accessibilité.
- <img src="logo.png" alt="Logo de l'entreprise">

3. Attributs pour les formulaires
   - type : Définit le type d'entrée pour un élément <input> (texte, mot de passe, email, etc.).
   - <input type="text" placeholder="Entrez votre nom">
   - value : Définit la valeur par défaut d'un champ de saisie ou d'un bouton.
   - <input type="text" value="Nom par défaut">
   - placeholder : Affiche un texte d'indication à l'intérieur d'un champ de saisie.
   - <input type="email" placeholder="Entrez votre adresse e-mail">
   - name : Donne un nom à un élément de formulaire, utilisé lors de l'envoi des données.
   - <input type="text" name="prenom">
   - required : Indique qu'un champ de formulaire est obligatoire.
   - <input type="password" required>
4. Attributs de style et de disposition
   - style : Permet d'ajouter du style en ligne (CSS) directement à un élément.
   - Texte bleu

Balises, Attributs

Balises, Attributs

Question 2 :

Qu’est est ce que c’est un attribut et quelles sont les types d’attributs qui existes?

- width et height : Définissent la largeur et la hauteur des éléments, souvent utilisés pour les images.
- <img src="image.jpg" width="200" height="100">

5. Attributs de données (data-*)
   - data-* : Attributs personnalisés pour stocker des données supplémentaires sur un élément HTML. Très utiles pour manipuler des éléments avec JavaScript.
   - Produit A
6. Attributs liés à l'accessibilité
   - aria-label : Fournit un label pour les éléments pour les rendre accessibles aux lecteurs d'écran.
   - <button aria-label="Fermer le menu">X</button>
   - role : Indique le rôle d'un élément dans la page (par exemple, button, navigation, alert).
   - Attention !

Les attributs sont essentiels en HTML pour enrichir les balises et fournir des informations supplémentaires ou des instructions spécifiques. Ils permettent d'ajuster le comportement et l'apparence des éléments, d'améliorer l'accessibilité, et d'interagir avec les scripts et les feuilles de style. Chaque attribut a une signification particulière et peut être utilisé pour rendre les pages web plus dynamiques, interactives et accessibles.

Balises, Attributs

Balises, Attributs

Question 2 :

Quelle est la structure de base d’un fichier HTML?

Premiers pas avec HTML

- Installation d'un éditeur de code : Comment installer et configurer un éditeur de code (comme VS Code ou Sublime Text) pour écrire du code HTML. Je vous conseil VSC.
- Créer votre premier fichier HTML : Création d’un simple fichier .html et affichage de celui-ci dans un navigateur.

Structure de base : Explication de la structure de base d'une page HTML avec les balises principales :

```
<!DOCTYPE html>
<html>
    <head>
        <title>Page de test</title>
    </head>
    <body>
        <h1>Hello, World!</h1>
    </body>
</html>
```

Balises, Attributs

Balises, Attributs

Question 3 :

Quelle sont les balises de mise en forme et de structuration de code?

1. Mise en forme du texte
   - Gras (<b>,)
   - Italique (<i>,)
   - Sauts de ligne (
) et espaces ( )
2. Structuration du contenu
   - Les sections (,)
   - Les balises de conteneur et de section (<header>, <footer>, <main>, <section>, <article>, <aside>)
3. Création de tableaux
   - Les balises de tableau (<table>, <tr>, <td>, <th>)
   - Fusionner des cellules (colspan, rowspan)

Balises, Attributs

Balises, Attributs

Question 4 :

Quelle sont les balises d’intégration et médias?

- Liens internes et externes

```
<a href="page2.html">Aller à la page 2</a>
<a href="https://www.example.com">Visitez Example</a>
```

- Liens d'ancrage

```
<a href="#section2">Aller à la section 2</a>
<h2 id="section2">Section 2</h2>
<a href="autrepage.html#section1">Aller à la section 1 de l'autre page</a>
```

- Liens vers des fichiers et ressources externes

```
<a href="document.pdf" download="MonDocument">Télécharger le PDF</a>
```

- Insertion d'images (<img>)

```
<img src="image.jpg" alt="Description" width="300" height="200">
```

- Intégration de vidéos et d'audios (<video>, <audio>)

```
<audio src="audio.mp3" controls> Votre navigateur ne supporte pas la lecture de fichiers audio. </audio>
```

Balises, Attributs

Balises, Attributs

Question 4 :

Quelle sont les balises d’intégration et médias?

- Utilisation des éléments source

```
<video controls width="500">
    <source src="video.mp4" type="video/mp4">
    <source src="video.webm" type="video/webm">
    Votre navigateur ne supporte pas la lecture de vidéo.
</video>
<audio controls>
    <source src="audio.mp3" type="audio/mpeg">
    <source src="audio.ogg" type="audio/ogg">
     Votre navigateur ne supporte pas la lecture audio.
 </audio>
```

Formulaire, Balise sémantique

Formulaires

Question 1 :

Qu’est ce qu’un formulaire et quelle est sa structure de base?

Les formulaires HTML permettent de collecter des informations auprès des utilisateurs et de les envoyer à un serveur pour traitement. Ce module explore la création de formulaires et leurs fonctionnalités avancées.

- Structure de base avec <form>

Explication : La balise <form> est utilisée pour créer un formulaire HTML. Elle définit le point de départ et la fin du formulaire, et permet de spécifier l'endroit où les données seront envoyées grâce à l'attribut action.

Exemple :

```
<form action="submit.php" method="post">
    <!-- Champs de formulaire ici -->
</form>
```

- Envoi des données : Introduction aux attributs method (GET et POST) et action pour comprendre comment les données sont envoyées à un serveur.

Formulaire, Balise sémantique

Formulaires

Question 2 :

Quelles sont les champs de saisie courants?

Types de <input> courants :

- Texte : <input type="text"> pour les noms, prénoms, etc.
- Mot de passe : <input type="password"> pour masquer les caractères.
- Email : <input type="email"> avec validation intégrée pour les adresses e-mail.
- Téléphone : <input type="phone"> avec validation intégrée pour les téléphone.
- Choix unique : <input type="radio"> pour faire un seul de choix réponse dans plusieurs réponses.
- Choix multiple : <input type="checkbox"> pour pouvoir fait des choix réponses dans plusieurs réponses.
- Numérique : <input type="number"> pour les chiffres seulement.

Exemple :

```
<form>
    <label for="name">Nom :</label>
    <input type="text" id="name" name="name">
    <label for="email">Email :</label>
    <input type="email" id="email" name="email">
</form>
```

Formulaire, Balise sémantique

Formulaires

Question 3 :

Comment implémente ont les boutons, zone de sélection et de zone de texte ?

- Boutons :

Création de boutons avec <button>, <input type="submit"> et <input type="reset">

```
<input type="reset" value="Réinitialiser">
<button type="reset">Réinitialiser</button>
<input type="submit" value="Envoyer">
<button type="submit">Envoyer</button>
```

- Menus déroulants avec <select> :

```
<label for="pays">Choisissez un pays :</label>
<select id="pays" name="pays">
<option value="benin">Bénin</option>
<option value="france">France</option>
<option value="canada">Canada</option>
</select>
```

- Zones de texte avec <textarea> :

```
<textarea name="message" rows="4" cols="50">Votre message ici...</textarea>
```

Formulaire, Balise sémantique

Formulaires

Question 4 :

Comment implémente ont les dates, time, et plage ?

Types avancés d'<input> :

- Date : <input type="date">
- Heure : <input type="time">
- Plage (range) : <input type="range"> pour sélectionner une valeur entre un minimum et un maximum.
- Couleur : <input type="color"> pour choisir une couleur.

Exemple :

```
<label for="date">Date de naissance :</label>
<input type="date" id="date" name="date">
<label for="time">L'heure de la réunion :</label>
<input type="time" id="time" name="time">
<label for="volume">Volume :</label>
<input type="range" id="volume" name="volume" min="0" max="100">
```

Formulaire, Balise sémantique

Formulaires

Question 5 :

Comment structure t-on un formulaire et comment valide t-on un formulaire?

Structuration avec <fieldset> et <legend> :
Explication : Utiliser <fieldset> pour regrouper des parties du formulaire et <legend> pour en indiquer l'objet.
Exemple :

```
<fieldset>
    <legend>Informations personnelles</legend>
    <label for="nom">Nom :</label>
    <input type="text" id="nom" name="nom">
    <label for="email">Email :</label>
    <input type="email" id="email" name="email">
</fieldset>
```

Validation simple avec HTML : Attributs de validation : required, pattern, min, max, maxlength, etc.
Exemple : `<input type="text" name="username" required pattern="[A-Za-z]{3,}">`

Formulaire, Balise sémantique

Balise sémentique

Question 6 :

Quelle sont les balises spécifique de HTML5 ?

Nouvelles balises HTML5 :

- <nav> : Délimite les zones de navigation.
- <figure> et <figcaption> : Utilisés pour insérer des images avec des légendes.
- <details> et <summary> : Permettent de créer des sections de contenu que l'utilisateur peut ouvrir et fermer.

Exemple :

```
<details>
    <summary>Plus d'informations</summary>
    <p>Voici du contenu supplémentaire caché par défaut.</p>
</details>
```

Balises sémantiques

Importance pour la lisibilité du code et l'accessibilité :

- SEO (référencement) : Les balises sémantiques aident les moteurs de recherche à comprendre la structure d'une page.
- Compatibilité et accessibilité : Facilite l'expérience des utilisateurs avec des handicaps, notamment grâce aux lecteurs d'écran.

Bonnes pratiques

Bonnes pratiques

Question 1 :

Quelle sont les bonnes pratiques de HTML5 ?

- Organisation et indentation :

Explication : Utiliser une structure d'indentation claire pour rendre le code plus lisible.

Exemple :

```
<nav>
    <ul>
        <li><a href="#home">Accueil</a></li>
        <li><a href="#about">À propos</a></li>
    </ul>
</nav>
```

- Commentaires HTML :

Syntaxe : <!-- Commentaire -->

Utilisation : Annoter le code pour mieux comprendre les sections complexes.

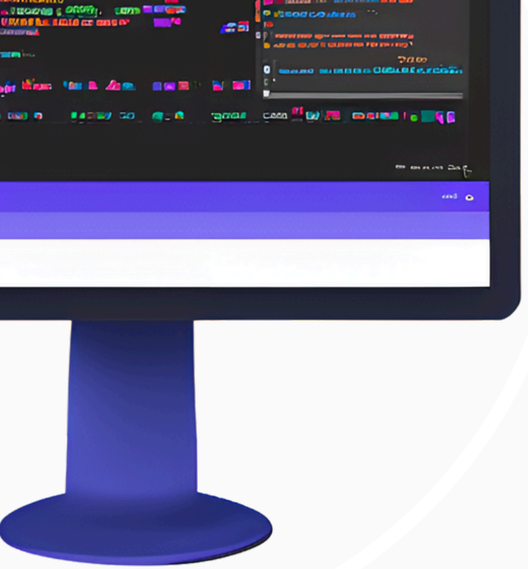
Exemple :

```
<!-- Ce lien mène à la page de contact -->
<a href="contact.html">Contact</a>
```

- Accessibilité avec les attributs ARIA :

Attributs ARIA : Introduction aux attributs comme aria-label, aria-hidden, role, etc.

Explication : Améliorer l'accessibilité des éléments pour les personnes utilisant des technologies d'assistance.



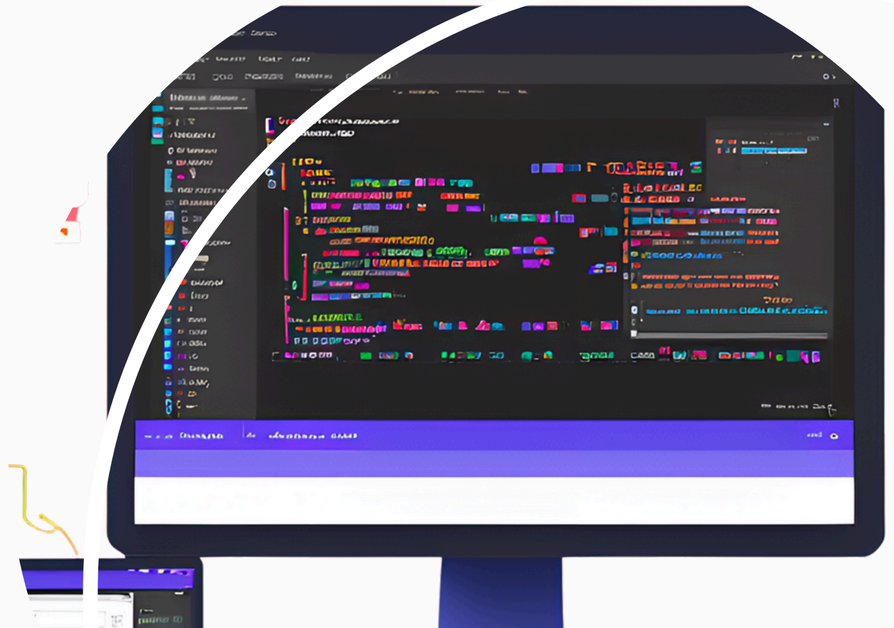






2 CSS

6h de discussion sur CSS



Plan CSS



Comprendre et connaître :

- Ce que c’est le CSS
- Différence entre HTML et CSS.
- Méthodes d'intégration de CSS
- Syntaxe de base
- Sélecteurs CSS
- Propriétés CSS de base
- Mise en page et disposition
- Styles avancés et design responsive
- Concepts avancés et bonnes pratiques CSS

Introduction

CSS et sa différence avec le HTML

Question 1 :

Qu’est-ce que c’est le CSS??

- Définition : CSS (Cascading Style Sheets) est un langage utilisé pour décrire la présentation d'une page web écrite en HTML. Il permet de contrôler l'apparence visuelle des éléments, comme la couleur, les polices, les marges, les positions, etc.
- Rôle : CSS sépare le contenu de la présentation, ce qui permet de styliser des pages de manière cohérente et efficace, de rendre les sites web plus attractifs et de faciliter la maintenance.

Question 2 :

Qu’est est la différence entre le CSS et le HTML?

- HTML est utilisé pour structurer le contenu d'une page web (par exemple, les titres, les paragraphes, les images).
- CSS est utilisé pour styliser ce contenu (par exemple, en ajoutant des couleurs, en ajustant la taille des polices, en créant des mises en page).
- Séparation des préoccupations : La séparation du HTML et du CSS facilite la conception et la gestion des sites web.

Intégrations, Syntaxes

Méthodes d’intégration du CSS

Question 1 :

Quelles sont les Méthodes d’intégration du CSS??

- Styles en ligne : Utiliser l'attribut style directement dans les balises HTML pour appliquer des styles spécifiques.

```
<p style="color: red;">Ce texte est en rouge.</p>
```

Avantages : Rapide à utiliser pour de petites modifications.

Inconvénients : Non recommandé pour les projets plus importants, car cela peut rendre le code difficile à maintenir.

- Styles internes :Ajouter du CSS dans une balise <style> placée dans la section <head> du fichier HTML.

```
<style>p {
   color: blue;
 }</style>
```

Avantages : Utile pour styliser une seule page.

Inconvénients : Moins efficace que les feuilles de style externes pour les sites multi-pages.

- Feuilles de style externes : Créer un fichier .css séparé et le lier au document HTML avec la balise <link>

```
<link rel="stylesheet" href="styles.css">
```

Avantages : Le moyen le plus efficace de gérer les styles pour plusieurs pages web. Facilite la maintenance et la cohérence des styles.

Intégrations, Syntaxes

Syntaxes de bases

Question 2 :

Quelles sont les deux syntaxes de bases en CSS??

- Sélecteurs, propriétés et valeurs
  - Sélecteurs : Utilisés pour cibler les éléments HTML à styliser (ex. p, .classe, #id).
  - Propriétés : Définissent ce qui doit être stylisé (ex. color, font-size).
  - Valeurs : Indiquent comment styliser la propriété (ex. red, 16px).

```
p {
    color: red;
    font-size: 16px;
}
```

- Commentaires CSS

Utilisés pour annoter le code et expliquer ce que font certaines parties :

```
/* Ceci est un commentaire */
```

Sélecteurs, Propriétés

Sélecteurs

Question 1 :

Qu’est-ce qu’un sélecteur en CSS??

Dans CSS, les sélecteurs sont utilisés pour cibler les éléments HTML à mettre en forme dans nos pages web. CSS propose une grande diversité de sélecteurs, offrant ainsi une précision très fine dans la façon de cibler les éléments.
Vous les avez déjà rencontrés, toute règle CSS commence par un sélecteur. Un sélecteur est une expression qui indique au navigateur à quelle entité HTML s'applique la règle CSS correspondante. Le ou les éléments ciblés par le sélecteur sont le sujet de ce sélecteur.

Sélecteurs, Propriétés

Sélecteurs

Question 2 :

Quelles sont les types sélecteurs de bases en CSS??

- Sélecteurs d'éléments

Sélectionnent tous les éléments du type spécifié, comme p, h1, div.

```
p {
 color: green;
}
```

- Sélecteurs de classe

Sélectionnent les éléments portant une classe spécifique (commence par un point).

```
 .highlight {
 background-color: yellow;
}
<p class="highlight">Texte surligné</p>
```

Sélecteurs, Propriétés

Sélecteurs

Question 2 :

Quelles sont les types sélecteurs de bases en CSS??

- Sélecteurs d'identifiant

Sélectionnent un élément unique avec un identifiant spécifique (commence par un dièse).

```
#header {
background-color: lightblue;
}
<div id="header">Entête</div>
```

- Sélecteurs universels

Sélectionnent tous les éléments avec étoile.

```
* {
    margin: 0;
    padding: 0;
}
```

Sélecteurs, Propriétés

Sélecteurs

Question 3 :

Quelles sont les autres types sélecteurs qui existent en CSS??

- Sélecteurs de descendant

Sélectionnent les éléments qui sont descendants d'autres éléments.

```
div p {
  color: purple;
}
```

- Sélecteurs de groupe

Permettent d'appliquer le même style à plusieurs sélecteurs.

```
h1, h2, h3 {
color: blue;
}
```

- Sélecteurs d'attribut

Sélectionnent les éléments ayant un attribut spécifique (et même une valeur spécifique).

```
input[type="text"] {
  border: 1px solid black;
}
```

Sélecteurs, Propriétés

Sélecteurs

Question 3 :

Quelles sont les autres types sélecteurs qui existent en CSS??

- Pseudo-classes

Ciblent un état spécifique d'un élément, comme lorsqu'il est survolé par la souris.

```
a:hover {
    color: red;
}
```

- Pseudo-éléments

Sélectionnent et stylisent une partie spécifique d'un élément, comme la première ligne ou avant/après un contenu spécifique.

```
p::first-line {
    font-weight: bold;
}
```

Sélecteurs, Propriétés

Sélecteurs

Question 3 :

Quelles sont les autres types sélecteurs qui existent en CSS??

- Utilisation de ::before et ::after pour ajouter du contenu.

```
p::before {
    content: "★ ";
    color: gold;
}

a::after {
    content: "★ ";
    color: green;
}
```

Sélecteurs, Propriétés

Propriétés

Question 1 :

Qu'est-ce qu'une propriété CSS ?

Une propriété CSS **détermine le style ou le comportement d'un élément HTML .** Les exemples incluent le style de police, la transformation, la bordure, la couleur et la marge. Une déclaration de propriété CSS se compose d'un nom de propriété et d'une valeur de propriété. Après deux points, la valeur est répertoriée après le nom de la propriété.

Question 2 :

Quelles sont les types de propriété CSS ?

Propriétés de text

- Couleur et police
  - color : Définit la couleur du texte.

```
p {
    color: blue;
}
```

Sélecteurs, Propriétés

Propriétés

Question 2 :

Quelles sont les types de propriété CSS ?

Propriétés de text

- font-family : Choix de la police de caractère, avec des polices de secours.

```
p {
    font-family: "Arial", sans-serif;
}
```

- font-size : Définit la taille de la police.

```
p {
    font-size: 16px;
}
```

- Styles de texte
  - font-style : Rend le texte en italique ou en normal (ex. italic, normal).

```
p {
    font-style: italic;
}
```

Sélecteurs, Propriétés

Propriétés

Question 2 :

Quelles sont les types de propriété CSS ?

Propriétés de text

- font-weight : Détermine l'épaisseur du texte (ex. bold, normal, 100, 900).

```
p {
    font-weight: bold;
}
```

- text-align : Aligne le texte (left, center, right, justify).

```
p {
    text-align: center;
}
```

- text-decoration : Ajoute des décorations au texte (underline, line-through, none).

```
a {
     text-decoration: underline;
 }
```

Sélecteurs, Propriétés

Propriétés

Question 2 :

Quelles sont les types de propriété CSS ?

Propriétés de text

- Espacement du texte
  - line-height : Contrôle l'espacement vertical entre les lignes de texte.

```
p {
    line-height: 1.5;
}
```

  - letter-spacing : Ajuste l'espacement entre les lettres.

```
p {
   letter-spacing: 0.1em;
}
```

  - word-spacing : Ajuste l'espacement entre les mots.

```
p {
   word-spacing: 0.5em;
}
```

Sélecteurs, Propriétés

Propriétés

Question 2 :

Quelles sont les types de propriété CSS ?

Propriétés d'arrière plan

- Couleurs
  - background-color : Définit la couleur de l'arrière-plan.
  - Valeurs de couleur : Hexadécimal (#ff0000), RGB (rgb(255, 0, 0)), RGBA (rgba(255, 0, 0, 0.5)), HSL (hsl(0, 100%, 50%)).

```
body {
  background-color: #f0f0f0;
}
```

- Images de fond
  - background-image : Ajoute une image de fond.

```
body {
  background-image: url("background.jpg");
}
```

Sélecteurs, Propriétés

Propriétés

Question 2 :

Quelles sont les types de propriété CSS ?

Propriétés d'arrière plan

- background-repeat : Contrôle la répétition de l'image de fond (repeat, no-repeat, repeat-x, repeat-y).

```
body {
  background-repeat: no-repeat;
}
```

- background-size : Définit la taille de l'image de fond (cover, contain, ou valeurs spécifiques).

```
body {
   background-size: cover;
}
```

- background-position : Positionne l'image de fond.

```
body {
  background-position: center;
}
```

Sélecteurs, Propriétés

Propriétés

Question 2 :

Quelles sont les types de propriété CSS ?

Propriétés d'espacement

- Marge ou espacement externe
  - margin : Crée de l'espace autour des éléments.

```
p {
    margin: 20px;
}
```

  - Propriétés spécifiques : margin-top, margin-right, margin-bottom, margin-left.
- Espacement interne
  - padding : Ajoute de l'espace à l'intérieur des éléments.

```
div {
    padding: 10px;
}
```

  - Propriétés spécifiques : padding-top, padding-right, padding-bottom, padding-left.

Sélecteurs, Propriétés

Propriétés

Question 2 :

Quelles sont les types de propriété CSS ?

Propriétés d'espacement

- Box Model

Comprendre le modèle de boîte qui définit comment un élément prend de l'espace : contenu, padding, border, margin.

  - Content – Le contenu de la boîte, où apparaissent le texte et les images
  - Padding – Efface une zone autour du contenu. Le remplissage est transparent
  - Border – Une bordure qui entoure le remplissage et le contenu
  - Margin – Efface une zone à l'extérieur de la bordure. La marge est transparente

  - box-sizing : Contrôle la manière dont la taille des éléments est calculée.

```
div {
    box-sizing: border-box;
}
```

Mise en page et disposition

Mise en page et disposition

Question 2 :

Quelles sont les propriété CSS de mise en page et de disposition ?

Propriétés de positionnement

- Types de positionnement
  - Positionnement statique : Valeur par défaut des éléments.

```
div {
    position: static;
}
```

  - Positionnement relatif : Positionné par rapport à sa position normale.

```
div {
    position: relative;
    top: 10px;
    left: 5px;
}
```

  - Positionnement absolu : Positionné par rapport au conteneur parent avec position: relative, absolute, ou fixed.

Mise en page et disposition

Mise en page et disposition

Question 2 :

Quelles sont les propriété CSS de mise en page et de disposition ?

Propriétés de positionnement

```
div {
    position: absolute;
    top: 20px;
    right: 10px;
}
```

- Positionnement fixe : Reste fixe même en défilement de page.
- Positionnement collant : Mélange entre relatif et fixe, basé sur le défilement de la page.

```
header {
    position: sticky;
    top: 0;
}
```

- Z-index : Contrôle la superposition des éléments.

```
div {
    position: absolute;
    z-index: 10;
}
```

Mise en page et disposition

Mise en page et disposition

Question 2 :

Quelles sont les propriété CSS de mise en page et de disposition ?

Propriétés de techniques de mise en page

- CSS Flexbox

Concepts de conteneur et éléments flexibles.

```
.container {
    display: flex;
    justify-content: center;
    align-items: center;
}
```

Disposition en ligne ou en colonne (flex-direction), gestion de l'espace (justify-content, align-items).

Mise en page et disposition

Mise en page et disposition

Question 2 :

Quelles sont les propriété CSS de mise en page et de disposition ?

Propriétés de techniques de mise en page

- CSS Grid

Création de grilles avec display: grid.

```
.grid-container {
    display: grid;
    grid-template-columns: 1fr 2fr;
    grid-gap: 10px;
}
```

Placement d'éléments dans des colonnes et des lignes, utilisation des propriétés grid-template-rows, grid-template-columns, grid-area.

Grilles dynamiques et structurées pour des mises en page complexes et réactives.

Concepts avancés

Animations et transitions CSS

Question 1 :

Comment faire des transitions en CSS?

Propriétés de transitions

Les transitions permettent de créer des effets de changement doux entre les styles d'un élément. Elles sont utiles pour améliorer l'interactivité et la fluidité des animations.

- transition-property : Spécifie la propriété à animer (ex. width, color, opacity).

```
div {
    transition-property: width;
}
```

- transition-duration : Définit la durée de la transition.

```
div {
    transition-duration: 0.5s;
}
```

- transition-timing-function : Contrôle la vitesse du changement pendant l'animation (ex. ease, linear, ease-in, ease-out).

Concepts avancés

Animations et transitions CSS

Question 1 :

Comment faire des transitions en CSS?

Propriétés de transitions

```
div {
    transition-timing-function: ease-in-out;
 }
 button {
    background-color: blue;
    transition: background-color 0.3s ease;
 }
 button:hover {
    background-color: red;
 }
```

Propriétés supplémentaires : animation-timing-function, animation-delay, animation-iteration-count (pour répéter l'animation).

Concepts avancés

Animations et transitions CSS

Question 2 :

Comment faire d’animations en CSS?

Propriétés d’ animations

Les animations permettent de définir des mouvements plus complexes en utilisant des étapes clés. Elles sont plus puissantes que les transitions simples.

- @keyframes : Décrit les étapes d'une animation.

```
@keyframes slideIn {
    from {
        transform: translateX(-100%);
    }
    to {
        transform: translateX(0);
    }
```

Concepts avancés

Animations et transitions CSS

Question 2 :

Comment faire d’animations en CSS?

Propriétés d’ animations

- animation-name et animation-duration : Applique l'animation aux éléments ciblés.

```
.box {
    animation-name: slideIn;
    animation-duration: 1s;
}
```

- Propriétés supplémentaires : animation-timing-function, animation-delay, animation-iteration-count (pour répéter l'animation).

Concepts avancés

Préprocesseurs CSS

Question 3 :

Comment manipuler les préprocesseurs en CSS?

1. Introduction aux préprocesseurs : LESS, Sass

- Les préprocesseurs CSS ajoutent des fonctionnalités supplémentaires à CSS qui simplifient le développement et la maintenance.
- LESS et Sass sont les deux préprocesseurs les plus populaires. Ils permettent l'utilisation de variables, de fonctions, et d'imbrication pour organiser et simplifier le code CSS.

2. Avantages des préprocesseurs CSS

- Variables : Simplifient les changements de couleurs, de polices, et d'autres propriétés répétées.

```
$primary-color: #3498db;
body {
    background-color: $primary-color;
}
```

- Mixins : Permettent de réutiliser des groupes de styles.

```
@mixin rounded-corners($radius) {
    border-radius: $radius;
}
```

Concepts avancés

Préprocesseurs CSS

Question 3 :

Comment manipuler les préprocesseurs en CSS?

```
.box {
        @include rounded-corners(10px);
  }
```

- Imbrication : Facilite la gestion des éléments enfants.

```
nav {
      ul {
        list-style: none;
       }
     li {
        display: inline-block;
       }
   }
```

- Modularité du code : Organise le CSS en fichiers plus petits, facilitant la maintenance.

Bonnes pratiques CSS

Bonnes pratiques

Question :

Quelles sont les bonne pratique à observer en CSS?

- Organisation et structure du CSS
  - Séparation des styles : Créez des fichiers CSS séparés pour les styles de base, les composants, les mises en page, et les animations.
  - Utilisation de commentaires : Ajoutez des commentaires pour décrire les sections importantes ou complexes du code.

```
/* Styles de navigation */
nav {
    background-color: #333;
}
```

  - Structure logique : Regroupez les styles similaires pour améliorer la lisibilité et la maintenance.
- Réduction du code redondant
  - Classes globales : Créez des classes réutilisables pour des styles communs (ex. margin-large, text-center).

```
.text-center {
    text-align: center;
}
```

Bonnes pratiques CSS

Bonnes pratiques

Question :

Quelles sont les bonne pratique à observer en CSS?

- Réutilisation des styles : Évitez de dupliquer le code en utilisant des classes communes pour appliquer les mêmes styles à plusieurs éléments.

- BEM (Block Element Modifier)

BEM est une méthodologie qui aide à organiser et à nommer les classes CSS de manière cohérente. Cela permet de rendre le code plus facile à comprendre et à maintenir.

- Block : Le composant principal.

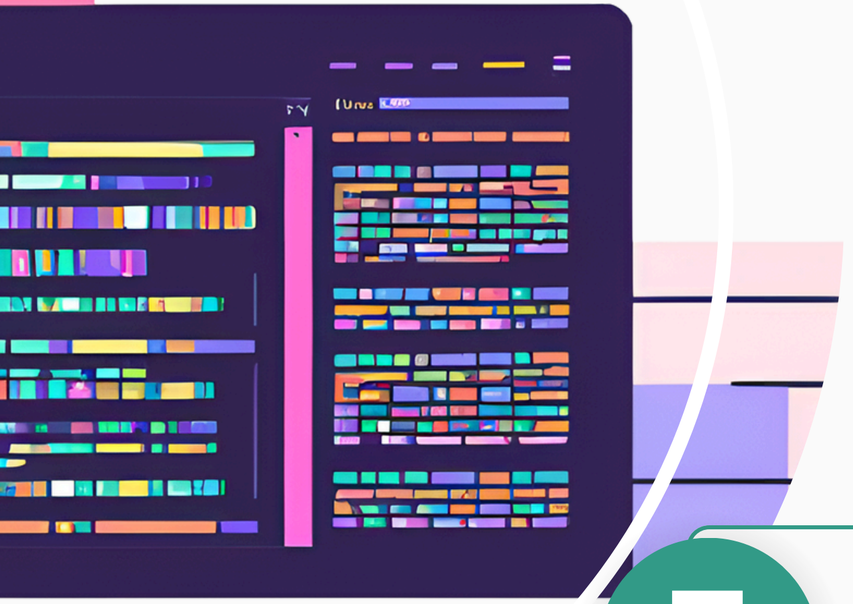
```
<div class="menu"></div>
```

- Element : Un sous-composant du bloc.

```
<div class="menu__item"></div>
```

- Modifier : Une variante ou un état du bloc ou de l'élément.

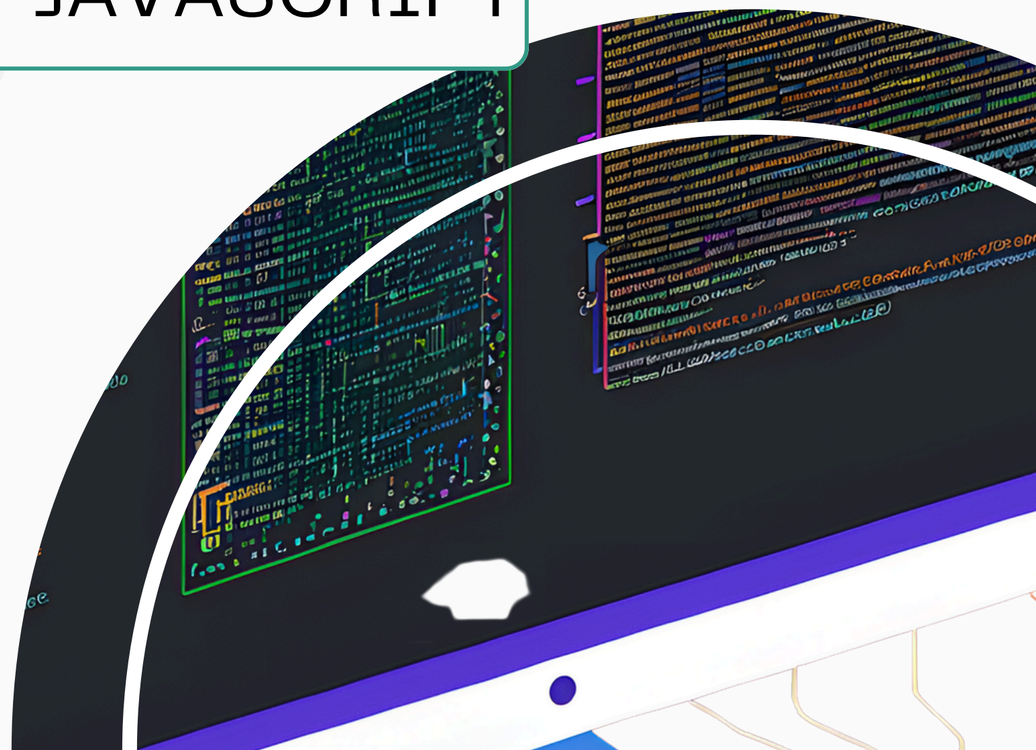
```
<div class="menu__item menu__item--active"></div>
```



3

JAVASCRIPT

6h de discussion sur JAVASCRIPT



Plan JAVASCRIPT



Comprendre et connaître :

- l'historique de JavaScript
- Variables, types de données et opérateurs
- Structures de contrôle
- Fonctions en JavaScript
- Le DOM (Document Object Model)
- Gestion des événements
- Manipulation des données avec JavaScript
- Concepts avancés de JavaScript
- Bonnes pratiques et outils de développement

Introduction

Historique de JavaScript

Question :

Qu’est-ce que c’est le JavaScript et quelle est son historique??

- Création et évolution
  - Origine : JavaScript a été créé par Brendan Eich en 1995, en seulement 10 jours, alors qu'il travaillait chez Netscape. Il était destiné à être un langage léger, facile à utiliser pour ajouter des fonctionnalités dynamiques aux pages web.
  - Initialement appelé Mocha : Puis il a été renommé LiveScript, et enfin JavaScript, pour capitaliser sur la popularité de Java à cette époque. Cependant, JavaScript et Java sont des langages distincts avec peu de choses en commun.
- Standardisation
  - ECMAScript (ES) : JavaScript a été standardisé sous le nom d'ECMAScript en 1997 par l'ECMA International. Depuis, plusieurs versions d'ECMAScript ont été publiées, introduisant de nouvelles fonctionnalités et améliorations.
  - ES6 (ECMAScript 2015) : Une mise à jour majeure qui a apporté des fonctionnalités importantes comme les classes, les modules, les promesses, et les let/const.

Introduction

Historique de JavaScript

Question :

Qu’est-ce que c’est le JavaScript et quelle est son historique??

- Évolution moderne
  - Mises à jour régulières : Chaque année, de nouvelles versions du langage sont publiées avec des améliorations. Des fonctionnalités telles que les async/await, les modules ES, et les améliorations de performance ont renforcé la popularité de JavaScript pour le développement web moderne.
  - Node.js : En 2009, JavaScript a été porté côté serveur grâce à Node.js, ce qui a permis aux développeurs d'écrire des scripts côté serveur avec JavaScript, transformant ainsi le langage en une technologie de bout en bout pour les applications web.

Manipulation de données

Variables, types de données et opérateurs

1

Question 1 :

Comment déclare t-on une variable en JavaScript??

Varables

- Déclaration de variables : var, let, et const.
  - var : Utilisé avant ES6, avec une portée de fonction. Désormais, let et const sont préférés.
  - let : Variable avec une portée de bloc. Permet la redéclaration dans des blocs différents.
  - const : Variable constante, sa valeur ne peut pas être réassignée.

```
let age = 23;
let nom = “SANNI”;
let prenom = “Olabissi”;
const pi = 3.14;
```

Manipulation de données

Variables, types de données et opérateurs

1.

Question 2 :

Quelles sont les type de données existant en JavaScript??

Type de données

- Types de données primitifs : string, number, boolean, null, undefined, et symbol.

```
let name = "Alice"; // string
let age = 30; // number
let isStudent = false; // boolean
let nothing = null; // null
let notAssigned; // undefined
```

- Objets et tableaux : Les objets peuvent contenir des paires clé-valeur et les tableaux stockent des listes d'éléments.

```
let person = { name: "John", age: 30 };
let numbers = [1, 2, 3, 4];
```

Manipulation de données

Variables, types de données et opérateurs

1

Question 3 :

Quelles sont les opérateurs les plus utilisés en JavaScript??

Opérateurs

- Opérateurs arithmétiques : +, –, *, /, %, ** (exponentiation).
  let sum = 10 + 5; // 15
  let power = 2 ** 3; // 8
- Opérateurs d'affectation : =, +=, –=, *=, /=.
  let x = 10;
  x += 5; // 15
- Opérateurs de comparaison : ==, ===, !=, !==, >, <, >=, <=.
  console.log(5 == "5"); // true (égalité)
  console.log(5 === "5"); // false (égalité stricte)
- Opérateurs logiques : && (ET), || (OU), ! (NON).
  let isAdult = age >= 18 && isStudent; // vrai si les deux conditions sont vraies

Manipulation de données

Manipulation de données

Question 1 :

Comment manipule t-on un tableau ??

Tableaux

- Définition et création : Les tableaux sont utilisés pour stocker plusieurs valeurs dans une seule variable.

```
let fruits = ["Apple", "Banana", "Cherry"];
```

- Méthodes de base : push(), pop(), shift(), unshift(), splice(), slice().

```
fruits.push("Orange"); // Ajoute "Orange" à la fin
let firstFruit = fruits.shift(); // Supprime et retourne "Apple"
```

- Itération sur les tableaux : for, forEach, map, filter, reduce.

```
fruits.forEach(fruit => console.log(fruit));
let fruitLengths = fruits.map(fruit => fruit.length);
```

Manipulation de données

Manipulation de données

Question 1 :

Comment manipule t-on un objet ??

Objets

- Définition et création : Les objets permettent de stocker des paires clé-valeur.

```
let person = {
    name: "Alice",
    age: 25,
    isStudent: true
};
```

- Accès et manipulation des propriétés : Utilisation des notations pointée et crochet.

```
console.log(person.name); // "Alice"
person.age = 26; // Mise à jour de l'âge
person["isStudent"] = false; // Mise à jour avec la notation crochet
```

Manipulation de données

Manipulation de données

Question 2 :

Comment manipule t-on une chaine de caractère et/ou un json ??

JSON , Chaine de caractères

- JSON (JavaScript Object Notation)
  - Introduction : Format de données léger utilisé pour échanger des données entre un client et un serveur.
  - Conversion JSON : JSON.stringify() et JSON.parse().

```
let jsonString = JSON.stringify(person); // Convertit l'objet en chaîne JSON
let parsedData = JSON.parse(jsonString); // Convertit la chaîne JSON en objet
```

- Manipulation des chaînes de caractères (Strings)
  - Méthodes de manipulation : length, toUpperCase(), toLowerCase(), split(), substring(), trim(), replace().

```
let message = " Hello, World! ";
let trimmed = message.trim(); // "Hello, World!"
let uppercased = message.toUpperCase(); // " HELLO, WORLD! "
```

DOM, Evenement

DOM

Question 1 :

Qu’est-ce que le DOM et comment accède t-on au éléments du DOM en JavaScript??

définition, objet dom, sélecteur

- Définition : Le DOM est une représentation arborescente des éléments HTML d'une page web. JavaScript peut interagir avec le DOM pour créer, modifier, ou supprimer des éléments dynamiquement.
- Concept d'objets : Chaque élément HTML (balises, attributs, contenu) devient un objet manipulable via JavaScript.
- Accéder aux éléments du DOM
  - Sélecteurs basiques : getElementById, getElementsByClassName, getElementsByTagName.

    ```
    let header = document.getElementById("main-header");
    let buttons = document.getElementsByClassName("btn");
    ```
  - Sélecteurs modernes : querySelector, querySelectorAll.

    ```
    let firstParagraph = document.querySelector("p");
    let allParagraphs = document.querySelectorAll("p");
    ```

DOM, Evenement

DOM

Question 1 :

Qu’est-ce que le DOM et comment accède t-on au éléments du DOM en JavaScript??

manipulation du dom

- Manipulation du DOM
  - Modifier le contenu : textContent, innerHTML, value.
- let title = document.getElementById("title");
- title.textContent = "New Title";
  - Changer les styles : style et classes CSS (classList).
- let element = document.querySelector(".box");
- element.style.backgroundColor = "blue";
- element.classList.add("active");
  - Ajouter ou supprimer des éléments : createElement, appendChild, remove.
- let newDiv = document.createElement("div");
- newDiv.textContent = "Hello!";
- document.body.appendChild(newDiv);

DOM, Evenement

Evenements

Question 2 :

Qu’est-ce qu’un évènement et comment écoute t-on les évènements en JavaScript??

définition et écouteurs

- Définition : Un événement est une action que l'utilisateur effectue sur la page (clic, survol, saisie clavier). JavaScript permet d'écouter ces actions et de réagir.
- Exemples courants : Clic, survol, saisie clavier, soumission de formulaire, etc.
- Écouteurs d'événements
  - Ajouter un écouteur : addEventListener

```
let button = document.getElementById("myButton");
    button.addEventListener("click", () => {
    alert("Button clicked!");
});
```

  - Types d'événements : "click", "mouseover", "keydown", "submit", etc.

```
window.addEventListener("scroll", () => {
    console.log("User is scrolling");
});
```

DOM, Evenement

Evenements

Question 2 :

Qu’est-ce qu’un évènement et comment écoute t-on les évènements en JavaScript??

- Manipulation des événements
  - Prévenir le comportement par défaut : event.preventDefault()

```
let link = document.querySelector("a");
link.addEventListener("click", (event) => {
    event.preventDefault(); // Empêche le lien de naviguer
    console.log("Navigation annulée");
});
```

  - Propagation des événements : event.stopPropagation()

```
let parentDiv = document.getElementById("parent");
parentDiv.addEventListener("click", () => {
    console.log("Parent clicked");
});
let childDiv = document.getElementById("child");
childDiv.addEventListener("click", (event) => {
    event.stopPropagation(); // Empêche la propagation de l'événement au parent
    console.log("Child clicked");
});
```

Structure de contrôle, Fonction

Structure de contrôle

Question 1 :

Comment implémente t-on les structures de contrôles en JavaScript??

le Si et le ternaire

- Instructions conditionnelles
  - if, else if, else : Permet d'exécuter du code basé sur une condition.

```
let age = 20;
if (age >= 18) {
    console.log("Adulte");
} else if (age >= 15) {
    console.log("Adonaissant");
} else {
    console.log("Enfant");
}
```

  - Opérateur ternaire : Version simplifiée de if-else.

```
let message = age >= 18 ? "Majeur" : "Mineur";
```

Structure de contrôle, Fonction

Structure de contrôle

Question 1 :

Comment implémente t-on les structures de contrôles en JavaScript??

le selon et le pour

- switch, case, default : Permet d'exécuter du code basé selon un choix.

```
switch (new Date().getDay()) {
    case 6:
      text = "Today is Saturday";
      break;
    case 0:
      text = "Today is Sunday";
      break;
    default:
      text = "Looking forward to the Weekend";
}
```

- Structures de boucle
  - Boucle for : Utilisée pour itérer un nombre connu de fois.

Structure de contrôle, Fonction

Structure de contrôle

Question 1 :

Comment implémente t-on les structures de contrôles en JavaScript??
le tant que et le repété

```
for (let i = 0; i < 5; i++) {
    console.log(i);
}
```

- Boucle while : Exécute tant qu'une condition est vraie.

```
let count = 0;
while (count < 5) {
  console.log(count);
  count++;
}
```

- Boucle do-while : Similaire à while, mais s'exécute au moins une fois avant de vérifier la condition.

```
let num = 0;
do {
  console.log(num);
  num++;
} while (num < 5);
```

Structure de contrôle, Fonction

Structure de contrôle

Question 1 :

Comment implémente t-on les structures de contrôles en JavaScript??

le saut

- Instructions de contrôle de boucle
  - break : Interrompt la boucle.

```
for (let i = 0; i < 10; i++) {
  if (i === 5) {
    break; // arrête la boucle à 5
  }
}
```

  - continue : Saute à la prochaine itération.

```
for (let i = 0; i < 10; i++) {
  if (i % 2 === 0) {
    continue; // saute les nombres pairs
  }
  console.log(i); // affiche seulement les nombres impairs
}
```

Structure de contrôle, Fonction

Fonction

Question 2 :

Quelle sont les types de fonction en JavaScript??
sans paramètre et avec.

- Introduction aux fonctions

Définition : Une fonction est un bloc de code réutilisable qui effectue une tâche spécifique. Les fonctions peuvent être définies une fois et appelées plusieurs fois, sans ou avec différents arguments.

```
function greet() {
    console.log("Hello, World!");
}
```

  - greet(); // Appel de la fonction sans argument

- Paramètres et arguments
  - Fonctions avec paramètres : Les fonctions peuvent accepter des valeurs d'entrée (paramètres).

```
function greet(name) {
    console.log("Hello, " + name + "!");
}
greet("Alice"); // Affiche "Hello, Alice!"
```

Structure de contrôle, Fonction

Fonction

Question 2 :

Quelle sont les types de fonction en JavaScript??
valeur de retour, anonyme et fléchée

- Valeurs de retour : Les fonctions peuvent renvoyer une valeur avec return.

```
function add(a, b) {
   return a + b;
}
let sum = add(5, 3); // 8
```

- Fonctions anonymes et expressions de fonction
  - Fonctions anonymes : Fonctions sans nom, souvent utilisées comme arguments dans d'autres fonctions.

```
const sayHello = function() {
     console.log("Hello!");
 };
sayHello();
```

  - Fonctions fléchées (arrow functions) : Syntaxe plus concise pour écrire des fonctions anonymes.

```
 const multiply = (a, b) => a * b;
 console.log(multiply(2, 3)); // 6
```

Structure de contrôle, Fonction

Fonction

Question 2 :

Quelle sont les types de fonction en JavaScript??
rappel soit callback

- Fonctions de rappel (callbacks)
  - Fonctions passées en paramètre : Une fonction peut être passée comme argument à une autre fonction, puis être appelée plus tard (asynchrone).

```
function performTask(callback) {
    console.log("Task in progress...");
    callback();
}
performTask(() => {
    console.log("Task completed!");
});
```

Concepts avancés, Bonnes pratiques

Concepts avancés de JavaScript

Question 1 :

Quelle sont les concepts avancés utils en JavaScript??

Scope et Closures

- Portée et fermeture (Scope & Closures)
  - Portée : Locale, globale, et bloc (let, const vs var).

```
function showName() {
    let name = "Alice"; // Variable locale
    console.log(name);
}
```

  - Closures : Une fonction qui se souvient de son contexte lexical, même si elle est exécutée hors de celui-ci.

```
function createCounter() {
    let count = 0;
    return function() {
        return ++count;
    };
}
let counter = createCounter();
console.log(counter()); // 1
console.log(counter()); // 2
```

Concepts avancés, Bonnes pratiques

Concepts avancés de JavaScript

Question 1 :

Quelle sont les concepts avancés utils en JavaScript??

fonction asynchrone

- Fonctions asynchrones
  - Les promesses (Promises) : Gestion des opérations asynchrones.

```
let promise = new Promise((resolve, reject) => {
    let success = true;
    if (success) resolve("Operation Successful");
    else reject("Operation Failed");
});
```

  - Async/Await : Syntaxe moderne pour écrire du code asynchrone de manière plus lisible.

```
async function fetchData() {
    let response = await fetch("https://api.example.com/data");
    let data = await response.json();
    console.log(data);
}
```

Concepts avancés, Bonnes pratiques

Concepts avancés de JavaScript

Question 1 :

Quelle sont les concepts avancés utils en JavaScript??

la POO

- Programmation orientée objet (POO)
  - Classes et objets : Définir des objets avec des méthodes et des propriétés.

```
class Person {
      constructor(name, age) {
          this.name = name;
          this.age = age;
       }
      greet() {
          console.log(` Hello, my name is ${this.name}` );
     }
}
   let john = new Person("John", 30);
   john.greet(); // "Hello, my name is John"
```

Concepts avancés, Bonnes pratiques

Concepts avancés de JavaScript

Question 1 :

Quelle sont les concepts avancés utils en JavaScript??

héritage

- Héritage : Créer des classes dérivées d'autres classes.

```
class Student extends Person {
    constructor(name, age, grade) {
        super(name, age);
        this.grade = grade;
    }
}
```

Concepts avancés, Bonnes pratiques

Bonnes pratiques en JavaScript

Question 2 :

Quelle sont les bonnes pratiques à savoir en JavaScript??

- Bonnes pratiques de codage
  - Nommer les variables et fonctions : Utiliser des noms descriptifs et cohérents (camelCase pour les variables et fonctions, PascalCase pour les classes).
  - Organisation et modularité : Diviser le code en fonctions plus petites et moduler en fichiers séparés.
  - Écrire du code DRY (Don't Repeat Yourself) : Éviter la redondance, favoriser la réutilisation de code.
- Débogage et gestion des erreurs
  - Outils de débogage : Console de navigateur, points d'arrêt (breakpoints), et outils de profilage.

```
console.log("Variable:", variable);
```

  - Gestion des exceptions : Utiliser try-catch pour capturer et gérer les erreurs.

```
try {
    let result = riskyOperation();
} catch (error) {
    console.error("An error occurred:", error);
}
```

Olabissi Marie-José SANNI

Merci pour votre attention et collaboration